

Métodos de Autenticación

siete dimensiones, no siete competidores

Guía de estudio del Módulo 05 — qué hace cada método, cómo está implementado en el backend FastAPI del repositorio, y cuál conviene según el contexto de seguridad.

1	Basic Auth	Las credenciales viajan en cada request
2	Session Based	El estado vive en el servidor, la cookie es el vehículo
3	Token Auth	Token opaco en header: una referencia sin significado
4	JWT	Token firmado y auto-contenido: nada que guardar
5	Cookie Based	El mismo JWT, pero en una cookie <code>httpOnly</code>
6	OAuth 2.0	El protocolo de delegación, no un formato de token
7	SSO / OIDC	Un login, muchas apps: el IdP guarda la password

La idea central: no hay un “mejor” método

La trampa de este tema es creer que los siete métodos compiten por el mismo puesto y que hay que coronar a uno. No es así. Son **dimensiones ortogonales** que se combinan.

Cuando alguien pregunta “¿JWT o sesiones?”, está mezclando dos ejes distintos. **JWT** responde a la pregunta *¿dónde vive el estado?* (en ningún lado: viaja dentro del token). **Cookie** responde a otra pregunta completamente distinta: *¿cómo viaja la credencial?* Por eso existen las cuatro combinaciones, y por eso este módulo tiene un método 4 (JWT en header) y un método 5 (el mismo JWT, en cookie).

LA PREGUNTA CORRECTA

No es “¿cuál método uso?” sino “¿qué combinación de dimensiones uso, dado mi cliente, mi tolerancia al estado y mi necesidad de revocar?”

Los cinco ejes del diseño

EJE	PREGUNTA QUE RESPONDE	VALORES TÍPICOS
Estado	¿Quién se acuerda de que estás logueado?	Stateless (el token lo prueba) · Stateful (el server lo recuerda)
Transporte	¿Cómo viaja la credencial hasta el server?	Header <code>Authorization</code> · Cookie · Body/form
Delegación	¿Quién verifica la password del usuario?	Tu propia app · Un tercero (IdP)
Duración	¿Cuánto vale la credencial una vez emitida?	Un request · Una sesión · Un token de largo plazo
Revocación	¿Podés cortar el acceso <i>ya mismo</i> ?	Sí, borrando del server · No, hasta que expire

Cómo se combinan en la práctica

Una app real no elige uno: usa varios a la vez, cada uno donde corresponde. SSO para los humanos de la organización, un token opaco revocable para las integraciones y los bots, y Basic detrás de la red interna para que el monitoreo consulte `/health`. Eso no es indecisión: es diseño guiado por el modelo de amenaza de cada cliente.

HONESTIDAD PRIMERO

Todo este módulo trata sobre **autenticación** (OWASP A07): *¿quién sos?* No cubre **autorización** (A01): *¿qué te dejo hacer?* Son problemas distintos y A01 es la brecha #1 del OWASP Top 10. Esa es la clase siguiente.

El mapa de los siete

Leé esta tabla de izquierda a derecha como una sola frase: “el método *N* guarda su estado en *X*, viaja por *Y*, y lo valida *Z*”. Las diferencias reales entre métodos están en esas tres columnas, no en el nombre.

#	MÉTODO	DÓNDE VIVE EL ESTADO	MECANISMO CLAVE
1	Basic Auth	En ningún lado — van las credenciales completas	Header <code>Authorization: Basic base64(user:pass)</code>
2	Session Based	En el servidor	Cookie <code>session_id</code> que apunta a una sesión guardada
3	Token Auth	En el servidor (token opaco)	Header <code>Bearer <token></code> que es una referencia
4	JWT	Stateless — todo va dentro del token	Token firmado y auto-contenido, verificado por firma
5	Cookie Based	Stateless — en la cookie	El mismo JWT, dentro de una cookie <code>httpOnly</code>
6	OAuth 2.0	Depende del formato del token	Protocolo: el cliente obtiene un token del authorization server
7	SSO / OIDC	En el IdP externo	Un login en el proveedor, muchas apps que confían en su token

Tres palabras que conviene no mezclar

PALABRA	QUÉ TIPO DE COSA ES	EJEMPLO
JWT	Un <i>formato</i> de token	Un string <code>header.payload.signature</code> firmado. Podés usarlo con o sin OAuth2.
Bearer	Un <i>esquema de transporte</i>	<code>Authorization: Bearer <loquesea></code> . No dice nada sobre qué hay adentro.
OAuth 2.0	Un <i>protocolo</i>	Define flujos y endpoints, pero no impone el formato del token.

POR ESO EL MÉTODO 3 Y EL 6 SE PARECEN TANTO

Token Auth (3) y OAuth2 password flow (6) usan el **mismo esquema de transporte** — `POST` con form al login, después `Bearer` en el header — y aun así son métodos distintos. Lo que cambia es **qué hay adentro del token**: en el 3 es un string opaco que el server busca en su almacén; en el 6 es un JWT firmado que el server verifica sin buscar nada.

Un experimento que vale más que la tabla

Con el backend levantado, abrí `GET /api/health`: muestra contadores vivos (`sessions_count`, `api_tokens_count`, `users_count`). Ahora logueate con cada método y mirá **qué contador se mueve**. Con sesiones y con token opaco, algo sube: el server está guardando estado. Con JWT — en header o en cookie — no se mueve nada. Ahí ves, en vivo, la diferencia entre stateful y stateless.

Estado: ninguno

Transporte: header

Valida: tu app, en cada request

Revocación: no aplica

Complejidad: ★

No hay login ni sesión: el cliente manda **email y password en todos y cada uno de los requests**, codificados en base64 dentro del header `Authorization`.

CLIENTE

Arma `base64(email:password)` en cada request

SERVER

Decodifica y verifica contra el hash Argon2id

RESPUESTA

200 con el perfil, o 401 con `WWW-Authenticate: Basic`

BASE64 NO ES CIFRADO

Es codificación, y se revierte en una línea: `echo -n "demo@ejemplo.com:demo12345" | base64 -d`. Quien vea el tráfico ve la password en texto plano. Por eso Basic **solo** es admisible sobre HTTPS, que es lo que cifra el canal — Basic no aporta confidencialidad propia.

En el proyecto

GET `/api/me/basic` — no hay endpoint de login: las credenciales van en el request protegido.

```
# app/methods/basic.py
def me_basic(credentials: Annotated[HTTPBasicCredentials, Depends(basic_scheme)]):
    user = authenticate_user(credentials.username, credentials.password)
    if user is None:
        # el header WWW-Authenticate es OBLIGATORIO en el 401 de Basic
        raise HTTPException(401, headers={"WWW-Authenticate": "Basic"})
    return user
```

DETALLE QUE SE VE LEYENDO EL CÓDIGO

Basic llama a `authenticate_user()` directamente, no a `verify_login()`. Es decir: **es el único método sin rate limiting**, porque el rate limit del módulo cuelga de los endpoints de login y Basic no tiene ninguno. Una razón más para no exponerlo a internet.

A FAVOR

- + Código mínimo y soporte universal: curl, navegadores, cualquier cliente HTTP.
- + Ideal para monitoreo (`/health`), herramientas internas y desarrollo local.
- + Sin estado que administrar ni almacén que escalar.

EN CONTRA

- La password circula en cada request: más exposición, más superficie de log.
- No hay logout, ni expiración, ni revocación granular.
- Para cortar el acceso hay que cambiar la password, lo que rompe todo lo demás.

REGLA DE ORO

Siempre sobre HTTPS. Nunca expuesto a internet. Y nunca con la password real de un usuario: si necesitas Basic para una integración, emít un **token de integración revocable** y usalo como "password".

2

Session Based

El clásico de la web · el estado vive en el servidor

Estado: **stateful (server)**

Transporte: **cookie**

Valida: **tu app, con lookup**

Revocación: **inmediata**

Complejidad: **★★**

Al loguearte, el server genera un **token opaco** —un string aleatorio sin significado—, lo guarda en su propio almacén y te lo devuelve en una cookie. En cada request lo busca ahí para saber quién sos.

1 · LOGIN

Verifica credenciales y crea la sesión

2 · GUARDA

```
sessions[token] =
  user_id en el server
```

3 · COOKIE

Responde con
session_id
httpOnly

4 · LOOKUP

Cada request: busca el token y resuelve el usuario

EL PUNTO QUE CASI TODOS CONFUNDEN

La cookie es apenas **el vehículo**: ese mismo `session_id` podría viajar en un header y seguiría siendo Session Based. Lo que define al método no es la cookie, es que **el estado vive en el servidor**.

En el proyecto

POST Body JSON. Verifica, crea la sesión y setea la cookie.
`/auth/session/login`

GET Lee la cookie, busca la sesión, devuelve el perfil.
`/api/me/session`

POST Borra la sesión del almacén: revocación inmediata.
`/auth/session/logout`

```
# app/methods/session_cookie.py - el login
user = verify_login(email, password)           # rate limit + timing attack
session_token = storage.create_session(user.id) # estado EN EL SERVER
response.set_cookie(
    key="session_id", value=session_token,
    httpOnly=True,           # el JS del navegador no puede leerla - mitiga XSS
    samesite="lax",          # no viaja en requests cross-site - mitiga CSRF
    secure=COOKIE_SECURE,    # false en dev (http), true en producción (HTTPS)
)
```

LA VENTAJA NÚMERO UNO

El logout **borra la sesión del almacén**. En ese mismo instante el token dejó de valer, aunque alguien lo hubiera copiado. Esto es exactamente lo que un JWT no puede hacer, y es la razón por la que las sesiones siguen vivas y sanas en 2026.

A FAVOR

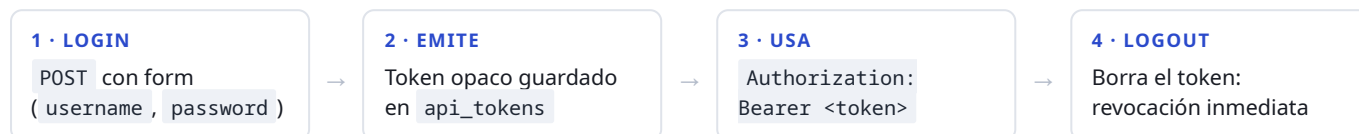
- + Revocación inmediata y control total desde el servidor.
- + Conceptualmente simple: una tabla o un diccionario de sesiones.
- + Natural para apps web renderizadas en el server.

EN CONTRA

- Hay estado que sostener: memoria compartida o Redis si hay varias instancias.
- Escalar la app implica escalar también el almacén de sesiones.
- Expuesto a CSRF y a session fixation si las cookies no están bien configuradas.

Estado: **stateful (server)**Transporte: **header Bearer**Valida: **tu app, con lookup**Revocación: **inmediata**Complejidad: **★★**

Misma filosofía que Session Based —el server guarda el estado— pero el token viaja en **Authorization: Bearer** en vez de una cookie. Es la autenticación clásica de las APIs, pensada para clientes que **no son un navegador**: apps móviles, otras APIs, CLIs.



“Opaco” quiere decir exactamente esto

El token es `secrets.token_urlsafe(32)`: 32 bytes aleatorios, criptográficamente seguros, **sin ninguna información adentro**. No podés leerlo, decodificarlo ni deducir nada de él. Es una llave de hotel: no dice quién sos, pero el server sabe a qué habitación corresponde. Esa opacidad es una virtud — no hay nada que filtrar si alguien lo intercepta y lo mira.

En el proyecto

POST `/auth/token/login` Form `urlencoded`, no JSON. Devuelve `{access_token, token_type: "bearer"}`.

GET `/api/me/token` Lee el header, busca el token en el almacén, resuelve el usuario.

POST `/auth/token/logout` Elimina el token del almacén.

```

# app/methods/token_bearer.py
user = verify_login(form_data.username, form_data.password, www_authenticate="Bearer")
opaque_token = storage.create_api_token(user.id) # estado EN EL SERVER
return Token(access_token=opaque_token, token_type="bearer")
  
```

¿POR QUÉ FORM Y NO JSON?

Porque es el mismo contrato que exige el password flow de OAuth 2.0. Así Swagger puede pedir un token desde su botón *Authorize* y probás el flujo completo en `/docs`. Fijate la lección: el **transporte es idéntico** al del método 6 — lo que cambia es qué hay dentro del token, opaco acá, JWT allá.

A FAVOR

- + Revocación inmediata, igual que las sesiones.
- + No depende de cookies, así que **no hay CSRF**: el header no se manda solo.
- + El cliente decide dónde guardarlo (keychain del móvil, por ejemplo).

EN CONTRA

- Requiere almacén y un lookup por request, igual que las sesiones.
- Los headers se filtran fácil en logs y proxies: cuidá qué registrás.
- Si el cliente es un navegador y lo guarda en `localStorage`, vuelve el riesgo de XSS.

4

JWT en header

Stateless · el token se prueba a sí mismo

Estado: **stateless**

Transporte: **header Bearer**

Valida: **la firma**

Revocación: **no, hasta expirar**

Complejidad: ★★★

El token deja de ser una referencia y pasa a ser **la credencial completa**: un JWT firmado que lleva la información adentro. El server no busca nada en ningún almacén — verifica la firma y con eso le alcanza.

PARTE	CONTENIDO	QUÉ SIGNIFICA
header	<code>{"alg": "HS256", "typ": "JWT"}</code>	Con qué algoritmo está firmado.
payload	<code>sub</code> , <code>iat</code> , <code>exp</code>	Quién es el usuario, cuándo se emitió y hasta cuándo vale.
signature	HMAC con la <code>SECRET_KEY</code>	La prueba de que el server lo emitió y nadie lo tocó.

EL PAYLOAD ES LEGIBLE

Está codificado en base64, **no cifrado**: cualquiera que tenga el token puede leer su contenido. La firma garantiza integridad y origen, no confidencialidad. Nunca metas adentro passwords ni datos sensibles: solo lo mínimo para identificar.

En el proyecto

POST `/auth/jwt/login` Body JSON. Devuelve el JWT firmado. No guarda absolutamente nada.

GET `/api/me/jwt` Decodifica, verifica firma y expiración, resuelve el usuario por `sub`.

```
# app/security.py — la verificación, con las dos defensas clave
return jwt.decode(
    token, SECRET_KEY,
    algorithms=[ALGORITHM],           # SIEMPRE explícito → anti algorithm confusion
    issuer=issuer, audience=audience, # validados en SSO (método 7)
    options={"require": ["sub", "exp"]}, # sin exp no hay token válido
)
```

POR QUÉ ALGORITHMS EXPLÍCITO NO ES OPCIONAL

Si dejás que el token diga con qué algoritmo verificarlo, un atacante lo cambia a `none` o fuerza una confusión entre simétrico y asimétrico, y se firma sus propios tokens. Es una de las vulnerabilidades clásicas de JWT, y se cierra con esa única línea.

A FAVOR

- + Escala horizontal sin almacén compartido: cualquier instancia valida sola.
- + Auto-contenido: los claims viajan con el token y no hay lookup.
- + Ideal entre microservicios, cada uno verificando `iss` y `aud`.

EN CONTRA

- **No se puede revocar** antes de que expire, salvo con una deny-list — que reintroduce el estado que quisiste evitar.
- Payload legible por cualquiera.
- Fácil de implementar mal: exige librería seria y validación estricta.

La **mitigación práctica** del problema de revocación es reducir `exp` (15–30 minutos) y sumar un refresh token rotado. Así una fuga dura minutos, no horas.

Estado: stateless

Transporte: cookie

Valida: la firma

Revocación: no, hasta expirar

Complejidad: ★★★

Este método existe para probar una sola cosa: **cambiar el vehículo cambia el modelo de amenaza**. El token es idéntico al del método 4, la lógica de verificación también. Solo cambia dónde viaja — y con eso cambian los ataques que te preocupan.

El tradeoff, que es el corazón del método

DÓNDE GUARDÁS EL TOKEN	QUÉ ATAQUE MITIGÁS	QUÉ ATAQUE TE QUEDA
Header + <code>localStorage</code> (lo típico en una SPA)	CSRF: el header no se manda solo, hay que ponerlo a mano.	XSS: cualquier script inyectado lee <code>localStorage</code> y se lleva el token.
Cookie <code>httpOnly</code> (este método)	XSS: el JavaScript no puede leer la cookie, ni el tuyo ni el inyectado.	CSRF: el navegador la manda sola en requests cross-site. Se mitiga con <code>SameSite</code> + token anti-CSRF.

NO HAY UNA OPCIÓN "MEJOR"

Elegís *qué ataque preferís enfrentar*. Para una app de navegador, la cookie `httpOnly` suele ganar: XSS es más común y más devastador, y CSRF ya tiene mitigaciones estándar y bien entendidas (`SameSite`, token anti-CSRF, verificación de origen).

En el proyecto

POST `/auth/jwt-cookie/login` Body JSON. Setea la cookie `access_token`.

GET `/api/me/jwt-cookie` Lee la cookie y verifica la firma. Sin lookup.

POST `/auth/jwt-cookie/logout` Borra la cookie... y nada más. Lee el recuadro de abajo.

```
# app/methods/jwt_cookie.py – el logout, que enseña más que el login
def jwt_cookie_logout(response: Response):
    response.delete_cookie("access_token")
    return Message(message="Cookie eliminada (el JWT sigue válido hasta expirar)")
```

EL LOGOUT QUE NO DESLOGUEA DEL TODO

En el método 2 el logout **borra la sesión del server** y el token muere en el acto. Acá no hay nada que borrar: sacás la cookie del navegador, pero si alguien copió el JWT antes, **ese token sigue valiendo hasta su `exp`**. El endpoint lo dice literalmente, y está bien que lo diga: es el precio del stateless.

A FAVOR

- + Amigable con el navegador: el front no manipula ni guarda el token.
- + Combina lo stateless del JWT con la ergonomía de las cookies.

EN CONTRA

- Arrastra la misma limitación de revocación del JWT.
- Hay que sumar defensa anti-CSRF de forma explícita.

Estado: según el token

Transporte: form + header

Valida: el authorization server

Revocación: depende

Complejidad: ★★★

OAuth 2.0 no reemplaza a los métodos anteriores: los **orquesta**. Define cómo un cliente obtiene un token y cómo lo usa, sin decir nada sobre qué hay adentro.

TRES PIEZAS QUE SE COMBINAN

OAuth 2.0 es el *protocolo*: los flujos, los grant types, el endpoint `/token`. · **JWT** es el *formato* del token — con tokens opacos seguiría siendo OAuth2. · **Bearer** es el *transporte*. Este método usa los tres a la vez, y esa es toda la novedad frente al método 3.

La idea que justifica su existencia: delegación

El problema original de OAuth es: *quiero que una app acceda a mis datos de otro servicio sin darle mi password*. La app nunca ve tus credenciales: recibe un token acotado, con permisos limitados y vencimiento, revocable sin tocar nada más.

Los grant types, y cuál usa el proyecto

FLOW	PARA QUÉ SIRVE	ESTADO EN 2026
Authorization code + PKCE	Apps de terceros, SPAs, móviles	El estándar recomendado: es lo que hay detrás de “Login with Google”.
Client credentials	Máquina a máquina, sin usuario	Vigente y correcto para servicios entre sí.
Password (el del proyecto)	Apps de primera parte, cliente de confianza	Desaconsejado salvo que cliente y server sean tuyos. Acá se usa por su valor didáctico.

MALA PRAXIS CATASTRÓFICA

Usar el **password flow con un tercero**. Si le pasás tu usuario y password de Google a una app cualquiera, destruiste justamente aquello para lo que OAuth fue inventado: solo tiene sentido cuando el cliente y el servidor son la misma organización.

En el proyecto

POST Form OAuth2 (`username` , `password` , `grant_type`) → { `access_token` , `token_type` } .
`/auth/oauth2/token`

GET Verifica el JWT por firma. Stateless, sin lookup.
`/api/me/oauth2`

```
# app/methods/oauth2.py – el contrato estándar del endpoint /token
async def oauth2_token_login(form_data: Annotated[OAuth2PasswordRequestForm, Depends()]):
    user = verify_login(form_data.username, form_data.password, www_authenticate="Bearer")
    access_token = security.create_access_token(subject=str(user.id))
    return Token(access_token=access_token, token_type="bearer")
```

Lo que el módulo deja fuera de alcance a propósito: el **refresh token con rotación**. El access token dura poco (acá 30 minutos) y el refresh, de vida larga, lo renueva sin volver a pedir la password. Es el siguiente paso real hacia producción.

Estado: en el IdP

Transporte: header o cookie

Valida: un tercero

Revocación: en el IdP

Complejidad: ★★★★★

SSO no es un método al mismo nivel que los otros seis: es una **experiencia** — un login en un proveedor de identidad y entrás a todas las apps que confían en él. Se *implementa* con OIDC (sobre OAuth 2.0) o SAML.

OIDC EN UNA LÍNEA

OAuth 2.0 responde “¿qué puede hacer esta app?”. OIDC le agrega el **id_token** —un JWT— para responder “¿quién es este usuario?”. es OAuth2 más una capa de identidad.

El flujo real, en siete pasos

- 1 El usuario entra a tu app y clickea “Login with IdP”.
- 2 Tu app redirige al IdP: `/authorize?client_id=...&redirect_uri=...`
- 3 El usuario se autentica **en el IdP** — o ya tiene sesión ahí, y eso es el SSO.
- 4 El IdP redirige de vuelta con un `code` corto y de un solo uso.
- 5 Tu app cambia ese code por tokens: `POST /token` con el `client_secret`.
- 6 El IdP devuelve el `id_token` (JWT con `iss` / `aud` / `sub` / `email`) y el `access_token`.
- 7 Tu app **valida** el `id_token` (firma, `iss`, `aud`, `exp`) y recién ahí confía.

En el proyecto (simulado, y honesto sobre qué simula)

POST `/api/sso/simulate` Actúa como el IdP: emite un `id_token` con claims OIDC, válido 5 minutos.

GET `/api/me/sso` Actúa como la app: valida el token del tercero y hace JIT provisioning.

```
# app/methods/sso_simulado.py – los tres checks obligatorios
payload = security.decode_token(
    credentials.credentials,
    issuer=SSO_ISSUER,          # ¿viene del IdP que espero?
    audience=SSO_CLIENT_ID,    # ¿fue emitido PARA mi app?
)                               # firma y exp los valida decode_token
```

LA PRUEBA QUE DEMUESTRA QUE LA VALIDACIÓN SIRVE

Un JWT emitido por *la misma app* en el método 4 es **rechazado** por `/api/me/sso` aunque la firma sea válida: le faltan el `iss` y el `aud` del IdP. Eso impide que un token emitido para otra aplicación sirva en la tuya.

Qué es real y qué no. Lo único simulado es la firma: se usa la clave propia en vez de la del IdP (en producción se verifica contra el **JWKS público** del proveedor). El resto es el patrón verdadero — claims OIDC estándar, checks de `iss` / `aud`, y **JIT provisioning**.

A FAVOR

- + Un login para todas las apps, con altas y bajas centralizadas.
- + El IdP maneja passwords, MFA y recuperación: vos no guardás credenciales.

EN CONTRA

- Dependés de un tercero, con su disponibilidad y sus decisiones.
- Validación estricta obligatoria: `iss`, `aud`, `exp`, `nonce`.

La matriz completa

Los siete métodos cruzados con las dimensiones que realmente los diferencian.

#	MÉTODO	ESTADO	TRANSPORTE	QUIÉN VALIDA	REVOCACIÓN	USO TÍPICO
1	Basic Auth	Stateless	Header <code>Basic</code>	La app, cada request	No aplica	APIs internas, monitoreo, primeros pasos
2	Session Based	Stateful	Cookie <code>session_id</code>	La app, con lookup	Inmediata	Web apps clásicas con login por formulario
3	Token Auth	Stateful	Header <code>Bearer</code>	La app, con lookup	Inmediata	APIs para móviles, SPAs, integraciones
4	JWT	Stateless	Header <code>Bearer</code>	La firma	Hasta expirar	APIs distribuidas, microservicios
5	Cookie Based	Stateless	Cookie <code>httpOnly</code>	La firma	Hasta expirar	SPAs y web apps con CSRF mitigado
6	OAuth 2.0	Según el token	Form + header	El authorization server	Depende	Delegación de acceso a recursos
7	SSO / OIDC	En el IdP	Header o cookie	El IdP externo	En el IdP	Login único entre apps, "Login with Google"

Qué pasa si te roban el token

Esta es, en la práctica, la pregunta que más define la elección.

MÉTODO	TU RESPUESTA ANTE EL ROBO
Sesiones / token opaco	Lo borras del almacén y el acceso muere en un segundo . Preferible cuando el riesgo es alto.
JWT (header o cookie)	No podés revocarlo. Solo te queda esperar el <code>exp</code> — por eso conviene que sea corto — o montar una deny-list.
Basic	No hay token: te robaron la password. Hay que cambiarla, y eso afecta todos los accesos del usuario.
SSO	Cortás la sesión en el IdP, que es el único lugar donde hay algo que cortar.

Recomendaciones por contexto

SI TU SITUACIÓN ES...	LA COMBINACIÓN RAZONABLE ES...
SPA + API propia, equipo chico	JWT en cookie <code>httpOnly</code> con token anti-CSRF; o JWT en header si la SPA lo mantiene solo en memoria.
App móvil + API propia	OAuth2 password flow + JWT de vida corta + refresh token rotado.
Web app clásica server-rendered	Session cookie + <code>SameSite</code> + token anti-CSRF.
Microservicios internos	JWT firmado por el gateway y validado en cada servicio, con <code>iss</code> y <code>aud</code> .
Empresa con varias apps	SSO/OIDC contra un IdP central (Keycloak, Okta, Entra ID).
Integraciones, bots, scripts	Token de integración opaco y revocable. Nunca Basic con la password real de una persona.

Las cinco preguntas, en orden

No empieces eligiendo un método. Contestá estas cinco preguntas y el método cae solo.

1 · ¿Quién es el cliente?

CLIENTE	LO QUE IMPLICA
Navegador (SPA o MVC)	Cookie <code>httpOnly</code> : el JavaScript no toca secretos.
App móvil	Header <code>Bearer</code> : no hay cookies. Token opaco o JWT en el keychain.
API hablando con API	Header <code>Bearer</code> o mTLS, con tokens cortos y rotación.
CLI o scripts	Basic sobre HTTPS, o mejor, un token de integración revocable.

2 · ¿Podés permitirte estado en el servidor?

TU NECESIDAD	LO QUE ELEGÍS
Varias instancias sin almacén compartido, o cero infraestructura de sesión	JWT (stateless)
Poder banear a alguien al instante	Sesiones o token opaco
Sesiones largas con “recordame”	Sesión con expiración deslizante

3 · ¿Quién tiene que conocer la password?

ESCENARIO	LO QUE ELEGÍS
Solo tu app, con credenciales propias	Basic, sesiones, tokens o JWT
Un tercero necesita acceso, pero no tu password	OAuth 2.0
Un login para todas las apps de la organización	SSO / OIDC

4 · ¿Qué riesgos trae el transporte que elegiste?

TRANSPORTE	RIESGO	MITIGACIÓN
Header <code>Authorization</code>	El token queda en logs y proxies	HTTPS siempre; nunca loguear headers de auth
Cookie <code>httpOnly</code>	CSRF	<code>SameSite</code> , token anti-CSRF, chequeo de origen
Body / form (OAuth2)	Exposición del <code>client_secret</code>	HTTPS, PKCE, secretos solo en clientes confidenciales

5 · ¿Qué hacés el día que se filtra un token?

Si es “*nada, hay que esperar*”, estás con un JWT y su `exp` tiene que ser corto. Si es “*lo borro del almacén*”, son sesiones o tokens opacos y el incidente se cierra en segundos. **Lo incorrecto es no haberte hecho la pregunta antes de elegir.**

Y PODÉS COMBINAR

Una misma app puede usar SSO para sus usuarios humanos, tokens opacos revocables para integraciones y Basic en la red interna para monitoreo. Eso no es inconsistencia: es diseño guiado por seguridad.

Buenas prácticas 2026

Estas decisiones no están solo explicadas: están **dentro del código** del backend de ejemplo. Son las que separan una demo de algo que podrías defender en producción.

#	PRÁCTICA	POR QUÉ
1	Argon2id para las passwords, vía <code>pwdlib</code>	Ganador del Password Hashing Competition. Nunca texto plano, nunca MD5 ni SHA1: son rápidos, y eso es exactamente lo que no querés en un hash de password.
2	<code>DUMMY_HASH</code> contra timing attacks	Cuando el email no existe, igual se corre un hash inútil. Así “ese usuario no existe” tarda lo mismo que “la contraseña está mal”, y nadie puede enumerar usuarios midiendo tiempos.
3	<code>algorithms=[ALGORITHM]</code> explícito	Cierra el ataque de algorithm confusion. Además se validan <code>iss</code> y <code>aud</code> en SSO y se exige la presencia de <code>sub</code> y <code>exp</code> .
4	Cookies <code>httpOnly</code> + <code>Secure</code> + <code>SameSite</code>	El JavaScript no puede leer el token, no viaja sobre HTTP en producción y no se manda en requests cross-site. <code>Secure</code> es config-driven vía <code>COOKIE_SECURE</code> .
5	Rate limiting del login	Cinco fallos por email en 15 minutos → 429 con <code>Retry-After</code> . Y el chequeo va <i>antes</i> de Argon2: un atacante no puede forzarte a gastar CPU en hashes caros.
6	Security headers globales	HSTS, <code>nosniff</code> , <code>X-Frame-Options</code> , <code>Referrer-Policy</code> , y <code>Cache-Control: no-store</code> en las rutas <code>/api/me/*</code> para que los perfiles no queden cacheados.
7	<code>SECRET_KEY</code> fail-loud	Con <code>ENVIRONMENT=production</code> , la app se niega a arrancar si no hay <code>SECRET_KEY</code> en el entorno. Sin defaults silenciosos: un secreto de desarrollo en producción es una puerta abierta.
8	Revocación donde hace falta	Sesiones y tokens opacos se revocan al instante. El JWT no, y el módulo lo asume como tradeoff intencional en vez de disimularlo.
9	Separación de capas	<code>security.py</code> no sabe nada de HTTP: lanza excepciones de dominio y la capa HTTP decide si eso es un 401 o un 403. La criptografía no debería conocer los status codes.

LO QUE ESTE MÓDULO NO CUBRE

Todo lo anterior es **autenticación**: OWASP A07, *¿quién sos?* Queda afuera la **autorización**: OWASP A01 — **Broken Access Control**, *¿qué te dejo hacer?*, que es la brecha número uno del Top 10. Un sistema puede autenticar perfecto y aun así dejar que un usuario lea los datos de otro cambiando un ID en la URL.

Lo que sigue: Nivel 03

Roles y permisos (RBAC), scopes de OAuth de verdad, **deny-by-default**, multi-tenancy e integración con un IdP real como Keycloak. Ahí es donde la autenticación que viste acá se convierte en control de acceso completo.

PARA LLEVARTE

Si de todo el módulo te queda una sola idea, que sea esta: **los siete métodos no compiten entre sí**. Cada uno ocupa un lugar distinto en un espacio de diseño de cinco ejes, y elegir bien es entender ese espacio — no memorizar cuál está de moda.